Assignment 6 — GenAI Code Comparison

1. Programs and access

I used ChatGPT through ChatGPT and Google Gemini through Gemini. I submitted the identical Assignment 6 prompt to both services with the inherited Assignment 5 C++ source.

2. Common prompt

I submitted the following exact prompt to both systems:

I am completing EECS 348 Software Engineering Assignment 6 at the University of Kansas. I have attached another student's Assignment 5 C++ Sudoku solver.

I need you to modify the existing program to meet the following requirements.

Requirements:
1. Preserve the original student's code structure, SudokuSolver class, recursive backtracking algorithm, and existing functionality. Do not completely rewrite the program.
2. Modify the program so that it reads five Sudoku puzzle files named puzzle1.txt through puzzle5.txt.
3. Instead of creating five separate solution files, write all solutions to a single file named solutions.txt.
4. For each puzzle, the output file must include the exact line "Puzzle: puzzleN.txt", where N is the puzzle number.
5. If solutions exist, write every valid Sudoku solution as a 9×9 grid with digits separated by spaces, matching the input puzzle format.
6. After the solutions for each puzzle, include the exact line "Solutions found: n", where n is the total number of solutions.
7. If no solutions exist, include the exact line "No solution found."
8. Maintain object-oriented programming principles and use recursive depth-first search with backtracking.
9. Include descriptive comments explaining the modifications, functions, and major sections of code.
10. Preserve the original author's information in the prologue comments. Add "Revised by:" and a revision date. Clearly identify AI-generated modifications in the comments.
11. Ensure the program compiles using g++ on a Linux server with C++17.
12. Include appropriate error handling for missing or invalid puzzle files.

Important constraints:
- Modify the provided Assignment 5 code rather than creating an entirely new implementation.
- Preserve as much of the original student's code as reasonably possible.
- Keep everything in one C++ source file named Assignment6.cpp.

- Do not use external libraries beyond the C++ standard library.
- Do not create a Makefile or additional source files.
- Preserve the original recursive backtracking functionality.
- Ensure all five puzzles are processed and their results are written to the same solutions.txt file.

Output requested:
First, provide the complete modified Assignment6.cpp source code.
After the code, explain which sections of the original program were changed and why.
Analyze your implementation in terms of:
1. Correctness
2. Execution time complexity
3. Space complexity
4. Maintainability
Do not omit any source code or replace sections with placeholders.

3. Correctness

I tested the ChatGPT-generated version locally with the five provided puzzle files. It recognizes underscores as blank cells and checks malformed inputs and conflicting clues. My reported solution counts were 1, 1, 1, 0, and 2 for puzzles 1–5, respectively.

When I reviewed the second implementation, I found that it uses “infile >> grid[i][j]” to read integers. Because my puzzle files use underscores for blank cells, that extraction fails. As written, this version cannot correctly load the supplied files. It also reports a different error message than the required no-solution line.

4. Execution time

Both versions use recursive depth-first search with backtracking. With E empty cells, I use O(9^E) as a worst-case search bound, with additional work for checking constraints. The ChatGPT version selects an empty cell with fewest candidates, while the second implementation searches in row-major order. I have not established a reliable runtime comparison from repeated measurements.

5. Space complexity

The ChatGPT implementation keeps a fixed 9×9 grid and a recursion stack with depth at most 81, streaming each solution to the file. Its auxiliary space is bounded for this fixed puzzle size. The second implementation saves every completed grid in a vector, requiring O(81S) storage for S solutions in addition to its board and recursion stack.

6. Maintainability

I found the ChatGPT version easier to maintain because it keeps the inherited SudokuSolver structure, input validation, console printing, and recursive search while changing the output

destination. The second version uses a different grid representation and stores solutions before writing, making it a larger departure from the inherited implementation.

7. Selection and justification

I selected the ChatGPT version as my baseline. It accepts underscore-based puzzle files, retains more of the inherited structure, validates the input, writes the required combined output, and avoids storing all solutions in memory. I chose it for correctness and maintainability, not because I demonstrated that it runs faster.

8. Improvements to the selected version

For my final version, I changed the per-puzzle output to a single stream opened in main(), added the required “Puzzle: ...” header for each input file, wrote each completed solution as it was found, and appended the appropriate count or no-solution message. I preserved the existing recursive search and documented my revisions.

9. Raw AI output — ChatGPT

Below is the complete source from the first AI implementation that I used as my baseline.

```
/*
Program: EECS 348 Assignment 6 - Recursive Sudoku Solver
Description: Modifies inherited Assignment 5 to write all five puzzle results to solutions.txt.
Inputs: puzzle1.txt through puzzle5.txt (9x9 whitespace-separated digits or underscores).
Outputs: Console solutions and one combined solutions.txt file.
Original author: Peter Lauman (Assignment 4, October 15, 2026).
Inherited Assignment 5 revision: Nathaniel Lanter (November 2, 2026).
Revised by: Peter Lauman, November 12, 2026.
Sources and attribution: inherited Assignment 5 revision by Nathaniel Lanter; original code by Peter Lauman;
ChatGPT assistance with Assignment 6 modifications.
I identified AI-assisted changes using [A6] comments and retained [ORIG] and [REV] markers to distinguish
inherited code.
Authorship: [ORIG] original code; [REV] inherited Assignment 5 revision;
[A6] modifications for Assignment 6 assisted by ChatGPT.
*/
#include <array>      // [ORIG] Fixed-size containers hold the nine-by-nine grid.
#include <fstream>    // [ORIG] File streams read puzzles; [REV] also used to write solution files.
#include <iostream>   // [ORIG] Standard output displays puzzles and solutions.
#include <ostream>    // [REV] Lets print() send a grid to either the console or a file.
#include <string>     // [ORIG] File names and error messages are strings.
#include <vector>     // [ORIG] Candidate lists store valid numbers for a cell.

class SudokuSolver {
private:
    std::array<std::array<int, 9>, 9> grid{}; // [ORIG] Zero represents an empty square.
    unsigned long long solutions = 0;        // [ORIG] Track how many completed grids are found.
    std::ostream* solutionFile = nullptr;     // [A6] Shared output stream owned by main().

    // [ORIG] Check whether a number violates any Sudoku constraint at a given square.
    bool allowed(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) { // [ORIG] Scan the corresponding row and column.
            if (grid[row][i] == value || grid[i][col] == value) return false; // [ORIG] Digit already in
row/column.
        }
        int firstRow = (row / 3) * 3; // [ORIG] Find the top row of the containing block.
        int firstCol = (col / 3) * 3; // [ORIG] Find the left column of the containing block.
        for (int r = firstRow; r < firstRow + 3; ++r) { // [ORIG] Visit each block row.
```

```
            for (int c = firstCol; c < firstCol + 3; ++c) { // [ORIG] Visit each block column.
                if (grid[r][c] == value) return false; // [ORIG] Reject repeated digits.
            }
        }
        return true; // [ORIG] No conflict was found.
    }

    // [ORIG] Use the empty square with the fewest legal candidates to reduce branching.
    bool chooseCell(int& bestRow, int& bestCol, std::vector<int>& bestValues) const {
        bestRow = -1; // [ORIG] Negative row means the board has no empty cells.
        bestCol = -1; // [ORIG] Negative column also indicates no choice yet.
        bestValues.clear(); // [ORIG] Remove candidates from the previous recursion level.
        for (int r = 0; r < 9; ++r) { // [ORIG] Inspect all rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Inspect all columns.
                if (grid[r][c] != 0) continue; // [ORIG] Ignore already filled cells.
                std::vector<int> candidates; // [ORIG] Store legal values for this square.
                for (int v = 1; v <= 9; ++v) { // [ORIG] Try digits one through nine.
                    if (allowed(r, c, v)) candidates.push_back(v); // [ORIG] Keep legal digits.
                }
                if (candidates.empty()) { // [ORIG] An empty square has no valid digit.
                    bestRow = r; // [ORIG] Identify the contradictory square.
                    bestCol = c; // [ORIG] Identify its column.
                    bestValues.clear(); // [ORIG] Signal that this branch cannot work.
                    return true; // [ORIG] A blank cell exists, but this branch is impossible.
                }
                if (bestRow == -1 || candidates.size() < bestValues.size()) { // [ORIG] Prefer fewer options.
                    bestRow = r; // [ORIG] Save the strongest next square.
                    bestCol = c; // [ORIG] Save its column.
                    bestValues = candidates; // [ORIG] Save its available digits.
                }
            }
        }
        return bestRow != -1; // [ORIG] False means the entire board is filled.
    }

    // [ORIG] Recursively explore every valid completion of the current grid.
    void backtrack() {
        int row, col; // [ORIG] Coordinates of the next empty square.
        std::vector<int> values; // [ORIG] Valid candidates for that square.
        if (!chooseCell(row, col, values)) { // [ORIG] No empty squares remain.
            ++solutions; // [ORIG] Count the completed Sudoku board.
            std::cout << "Solution " << solutions << ":\n"; // [ORIG] Label each solution on the console.
            print(std::cout); // [REV] Print the completed board to the console (print now takes a stream).
            print(*solutionFile); // [A6] Append completed board to shared solutions.txt.
            return; // [ORIG] Continue searching after returning to the caller.
        }
        for (int value : values) { // [ORIG] Explore each candidate in order.
            grid[row][col] = value; // [ORIG] Tentatively fill the chosen square.
            backtrack(); // [ORIG] Recursively solve the smaller puzzle.
            grid[row][col] = 0; // [ORIG] Undo the guess so other possibilities can be tried.
        }
    }

public:
    // [ORIG] Read exactly 81 tokens, allowing underscores for blank squares.
    bool load(const std::string& filename) {
        std::ifstream input(filename); // [ORIG] Open the named puzzle.
        if (!input) return false; // [ORIG] Report missing or unreadable files.
        grid = {}; // [ORIG] Reset the grid before reading another puzzle.
        solutions = 0; // [ORIG] Reset the solution counter.
        std::string token; // [ORIG] Hold each number or underscore.
        for (int r = 0; r < 9; ++r) { // [ORIG] Read nine rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Read nine cells per row.
                if (!(input >> token)) return false; // [ORIG] Reject short or malformed files.
                if (token == "_") grid[r][c] = 0; // [ORIG] Convert a blank to zero.
                else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') // [ORIG] Accept a single
digit 1-9.
                    grid[r][c] = token[0] - '0'; // [ORIG] Convert a digit to an integer.
```

```
                else return false; // [ORIG] Reject unexpected input.
            }
        }
        if (input >> token) return false; // [ORIG] Reject files containing extra cells.
        for (int r = 0; r < 9; ++r) { // [ORIG] Check all given values.
            for (int c = 0; c < 9; ++c) { // [ORIG] Check every square in each row.
                if (grid[r][c] == 0) continue; // [ORIG] Blank squares cannot conflict.
                int value = grid[r][c]; // [ORIG] Temporarily remove a given digit.
                grid[r][c] = 0; // [ORIG] Avoid comparing the given with itself.
                bool valid = allowed(r, c, value); // [ORIG] Test row, column, and block.
                grid[r][c] = value; // [ORIG] Restore the original given digit.
                if (!valid) return false; // [ORIG] Reject conflicting clues.
            }
        }
        return true; // [ORIG] The puzzle was read and validated.
    }

    // [REV] Print the grid to any output stream (console or file) in the same format as the input files.
    void print(std::ostream& out) const {
        for (const auto& row : grid) { // [ORIG] Iterate through the nine rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Print all nine columns.
                if (c) out << ' '; // [REV] Separate adjacent cells with spaces (now writes to "out").
                if (row[c] == 0) out << '_'; // [REV] Show blanks as underscores.
                else out << row[c]; // [REV] Show filled squares as digits.
            }
            out << '\n'; // [REV] End the current row.
        }
        out << '\n'; // [REV] Blank line separates grids visually.
    }

    // [REV] Solve the loaded puzzle and write all results to the named solution file.
    void solve(std::ostream& combinedOutput) { // [A6] Reuse one stream for all puzzles.
        solutionFile = &combinedOutput; // [A6] Set destination for recursive solution output.
        backtrack(); // [ORIG] Begin recursive depth-first search.
        if (solutions == 0) { // [ORIG] Handle failure.
            std::cout << "No solution found\n"; // [ORIG] Console message for unsolvable boards.
            (*solutionFile) << "No solution found.\n"; // [REV] File must contain exactly this single line.
        } else { // [REV] At least one solution was found.
            std::cout << "Total solutions: " << solutions << "\n"; // [ORIG] Report count on the console.
            (*solutionFile) << "Solutions found: " << solutions << "\n"; // [REV] Required final line in the
file.
        }
        solutionFile = nullptr; // [A6] main() owns and closes the combined output file.
    }
};

// [ORIG] Run the same object-oriented solver on all five assigned puzzle files.
int main() {
    std::ofstream combinedOutput("solutions.txt"); // [A6] Create one output file for all puzzles.
    if (!combinedOutput) { // [A6] Detect output file permission errors.
        std::cerr << "Unable to create solutions.txt\n"; // [A6] Explain file error.
        return 1; // [A6] Report unsuccessful execution.
    }
    for (int number = 1; number <= 5; ++number) { // [ORIG] Process every supplied puzzle.
        std::string filename = "puzzle" + std::to_string(number) + ".txt"; // [ORIG] Build file name.
        combinedOutput << "Puzzle: " << filename << "\n"; // [A6] Label each puzzle in the combined file.
        SudokuSolver sudoku; // [ORIG] Create a fresh object for this puzzle.
        std::cout << "=== " << filename << " ===\n"; // [ORIG] Identify the input file.
        if (!sudoku.load(filename)) { // [ORIG] Read and validate the puzzle.
            std::cout << "Unable to read puzzle or invalid starting grid.\n\n"; // [ORIG] Explain failure.
            combinedOutput << "No solution found.\n"; // [A6] Required result for invalid puzzle.
            continue; // [ORIG] Move on to the next puzzle file.
        }
        std::cout << "Original puzzle:\n"; // [ORIG] Label the given grid.
        sudoku.print(std::cout); // [REV] Show the initial board (print now takes a stream).
        sudoku.solve(combinedOutput); // [A6] Append solutions and count to combined output.
        std::cout << '\n'; // [ORIG] Separate results from different puzzles.
    }
```

```
    return 0; // [ORIG] Finish successfully.
}
```

10. Raw AI output — Google Gemini

The following is the Gemini C++ code I received. I have preserved the code as generated, including its input-format limitation and placeholder attribution. The Gemini response did not include a separate written explanation in the material I retained.

```
/**
 * @file Assignment6.cpp
 * @brief EECS 348 Software Engineering Assignment 6 - Sudoku Solver
 * @original_author Original Assignment 5 Student Author
 * @revised_by AI Collaborator
 * @revision_date November 12, 2026
 * @details AI-Generated Modifications:
 *          - Extended the single-puzzle logic to process a batch of five files (puzzle1.txt through
puzzle5.txt).
 *          - Consolidated output redirection from individual solution files to a single unified "solutions.txt"
file.
 *          - Implemented formatting rules including exact header lines ("Puzzle: puzzleN.txt"), grid spacing,
 *            and solution counts ("Solutions found: n" or "No solution found.").
 *          - Added comprehensive error handling for missing or malformed input files.
 *          - Preserved the original object-oriented class structure and recursive depth-first backtracking
algorithm.
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    int grid[9][9];
    std::vector<std::vector<int>> solutions;

    /**
     * @brief Validates if placing a number at a given cell satisfies Sudoku rules.
     * @param r Row index (0-8)
     * @param c Column index (0-8)
     * @param num Number to place (1-9)
     * @return true if valid, false otherwise
     */
    bool isValid(int r, int c, int num) {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[r][i] == num) return false;
            if (grid[i][c] == num) return false;
        }

        // Check 3x3 subgrid constraints
        int startRow = r - r % 3;
        int startCol = c - c % 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == num) return false;
            }
        }
        return true;
    }

    /**
     * @brief Recursive depth-first search helper with backtracking to find all valid solutions.
     * @param r Current row index
```

```
     * @param c Current column index
     */
    void solveHelper(int r, int c) {
        // Base case: If we have successfully filled past the last row
        if (r == 9) {
            std::vector<int> currentSolution;
            currentSolution.reserve(81);
            for (int i = 0; i < 9; ++i) {
                for (int j = 0; j < 9; ++j) {
                    currentSolution.push_back(grid[i][j]);
                }
            }
            solutions.push_back(currentSolution);
            return;
        }

        // Determine next cell coordinates
        int nextRow = (c == 8) ? r + 1 : r;
        int nextCol = (c == 8) ? 0 : c + 1;

        // If cell is already pre-filled, move to the next cell
        if (grid[r][c] != 0) {
            solveHelper(nextRow, nextCol);
        } else {
            // Try digits 1 through 9
            for (int num = 1; num <= 9; ++num) {
                if (isValid(r, c, num)) {
                    grid[r][c] = num;
                    solveHelper(nextRow, nextCol);
                    grid[r][c] = 0; // Backtrack
                }
            }
        }
    }

public:
    /**
     * @brief Construct a new Sudoku Solver object and initialize grid to zeros.
     */
    SudokuSolver() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                grid[i][j] = 0;
            }
        }
    }

    /**
     * @brief Loads a 9x9 Sudoku puzzle from a text file.
     * @param filename Name of the input puzzle file
     * @return true if loaded successfully, false otherwise
     */
    bool loadPuzzle(const std::string& filename) {
        std::ifstream infile(filename);
        if (!infile.is_open()) {
            return false;
        }
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (!(infile >> grid[i][j])) {
                    return false;
                }
            }
        }
        return true;
    }

    /**
```

```
     * @brief Initiates the solving process.
     */
    void solve() {
        solutions.clear();
        solveHelper(0, 0);
    }

    /**
     * @brief Retrieves all solutions discovered during execution.
     * @return const std::vector<std::vector<int>>& Reference to vector of solutions
     */
    const std::vector<std::vector<int>>& getSolutions() const {
        return solutions;
    }
};

int main() {
    // Open unified output file
    std::ofstream outfile("solutions.txt");
    if (!outfile.is_open()) {
        std::cerr << "Error: Could not open solutions.txt for writing.\n";
        return 1;
    }

    // Loop through puzzle1.txt through puzzle5.txt
    for (int n = 1; n <= 5; ++n) {
        std::string filename = "puzzle" + std::to_string(n) + ".txt";
        outfile << "Puzzle: puzzle" << n << ".txt\n";

        SudokuSolver solver;
        if (!solver.loadPuzzle(filename)) {
            outfile << "Error: Could not open or read " << filename << ".\n";
            outfile << "Solutions found: 0\n";
            continue;
        }

        solver.solve();
        const auto& solutions = solver.getSolutions();

        if (solutions.empty()) {
            outfile << "No solution found.\n";
        } else {
            // Write each valid solution grid
            for (const auto& sol : solutions) {
                for (int i = 0; i < 81; ++i) {
                    outfile << sol[i] << ((i % 9 == 8) ? "" : " ");
                    if (i % 9 == 8) {
                        outfile << "\n";
                    }
                }
            }
            outfile << "Solutions found: " << solutions.size() << "\n";
        }
    }

    outfile.close();
    std::cout << "Assignment 6 completed successfully. Results saved to solutions.txt.\n";
    return 0;
}
```